
Query Optimization: Hints

This lab serves as the second part of our previous lab, where we began exploring database optimization techniques. In this part, we will focus on Hints as a powerful tool for influencing the execution plan of SQL queries. Hints allow the provision of the database engine with specific instructions on how to optimize the query execution, helping to improve performance in certain scenarios. At the end of this lab, we will explore how *Indexes*, *Materialized Views*, and *Hints* are all interrelated. These concepts contribute collectively to the optimization of database queries, enhancing their efficiency and performance.

① Useful Notions & Syntaxes

(a) Hint

- i) *Hints* are instructions that can be added to a SQL statement to indicate (The optimizer) the most efficient way to run the statement.
- ii) Hints are written as a special kind of comment that must be placed immediately after the *SELECT* keyword. What distinguishes a hint from a regular comment is the " + " character immediately after (without any space) the opening " / * ". The syntax is as follows:

```
-----Creating Hints
SELECT /*+ HINT_NAME([parameters]) */ column1, column2
FROM table_name
WHERE condition;
```

- iii) There are several types of hints in database optimization, such as *JOIN* (*Specifies the type of join*); *USE_HASH* (*Forces the optimizer to use a hash join instead of other join types.*); ..., among others. Among them, the *ALL_ROWS* hint optimizes for efficient full query execution, making it ideal for batch processing. The *FIRST_ROWS* hint prioritizes fetching the initial rows quickly, which is useful for interactive queries. Lastly, the *INDEX* hint forces the optimizer to use a specified index, helping to speed up queries by avoiding full table scans.
- iv) The *ALL_ROWS* hint instructs the optimizer to choose a plan that minimizes the overall execution time for retrieving all rows of a query. It prioritizes throughput (that is, minimum total resource consumption) over response time. The syntax is as follows:

```
SELECT /*+ ALL_ROWS */ COLUMN1, COLUMN2, ...
FROM Table_Name
WHERE Condition;
```

- v) The *FIRST_ROWS*(*n*) hint tells the optimizer to prioritize returning the first *n* rows as quickly as possible. It optimizes for response time rather than total execution time. We can use *FIRST_ROWS*(*n*) or *FIRST_ROWS*. The last one still tells the optimizer to focus on fetching the first rows quickly, but it lets the database decide how many rows to prioritize based on internal heuristics. The syntax is as follows:

```
SELECT /*+ FIRST_ROWS(n) */ COLUMN1, COLUMN2, ...  
FROM Table_Name  
WHERE Condition;
```

- vi) An Index hint explains how the optimizer scans a specified index rather than a table.
vii) The hint can suggest the application of a specific index that is already created on the specified table. In this case, the hint is providing a directive to the query optimizer, indicating that it should consider using the specified index when executing the query, as:

```
----The syntax for the Index hint  
index(table_name,index_name)
```

```
SELECT /*+ INDEX(table_name index_name) */ column1, column2  
FROM table_name  
WHERE condition;
```

② Lab Work

- i) Display the execution plan using the *EXPLAIN PLAN*, by employing the different types of hints. **Analyze and clarify the findings.**

Queries

- i) Find the average rating for each show along with the total number of votes.
- ii) Find shows where the average rating is higher than the overall average rating.
- iii) *Find staff members who have worked on shows with the highest average rating.*
- iv) Find the top N staff members with the highest average rating across all shows. Where N is between 20 and 200.
- v) Find the staff member with the highest average rating for shows in a specific genre (Short, tvSeries, tvMovie).
- vi) Find the staff member with the highest average rating for shows in each genre.
- vii) Find shows that have more than one director.
- viii) Find the staff member who has worked in the most diverse set of categories (e.g., director, actor, etc.).
- ix) Find the top N episodes with the highest average rating and the most number of votes. Where N is between 5 and 10.
- x) Find shows that have consistently maintained an average rating above 6.5 across multiple seasons.

- xi) Find the average rating trend for each episode in a show by comparing it with the average rating of the previous episode.
- xii) Create a matrix showing the number of times each pair of staff members has collaborated in the same show.
- xiii) Find shows where the number of distinct staff members (across all categories) is above 5.

③ Homework

The queries that you cannot finish at the lab session are considered homework.